

# 5. NEURAL NETWORKS MONITORING

Stephan Robert-Nicoud  
HEIG-VD/HES-SO

*Credit: Andres Perez-Urbe*



# Objectives

- ❑ Understand the effect of hyper-parameters in a model
- ❑ Understand how to monitor the training of a neural network during the hyper-parameter tuning process
- ❑ Apply the “generic” Machine-Learning model selection methodology to neural networks using training, validation, and test datasets

# Data-driven modeling methodology

- ❑ What sort of data do I have ? is it noisy ? do I have missing data ? outliers ? redundant variables ?
  - ❑ Exploratory analysis of data
  - ❑ variable selection, noise reduction, outlier filtering
- ❑ Do I have variables whose values are in very different ranges ?
  - ❑ Data normalization
- ❑ Do I have an unbalanced dataset (number of examples per class) ?



- ❑ Create a train-validation-test partition of the dataset
- ❑ Baseline of neural network performance
  - ❑ identify number of epochs and learning rate using a first ANN configuration
- ❑ Neural network model selection
  - ❑ cross-validation evaluation using different hyper-parameters (e.g., number of hidden neurons)
- ❑ Compute final performances using the test dataset

# Data normalization (1)

Data normalization is needed if variables have different ranges. The simplest way to deal with this problem is to re-scale inputs using:

$$x' = \frac{(x - x_{min})}{(x_{max} - x_{min})}$$

Rescaled into [0,1] range

$$x' = 2 \frac{(x - x_{min})}{(x_{max} - x_{min})} - 1$$

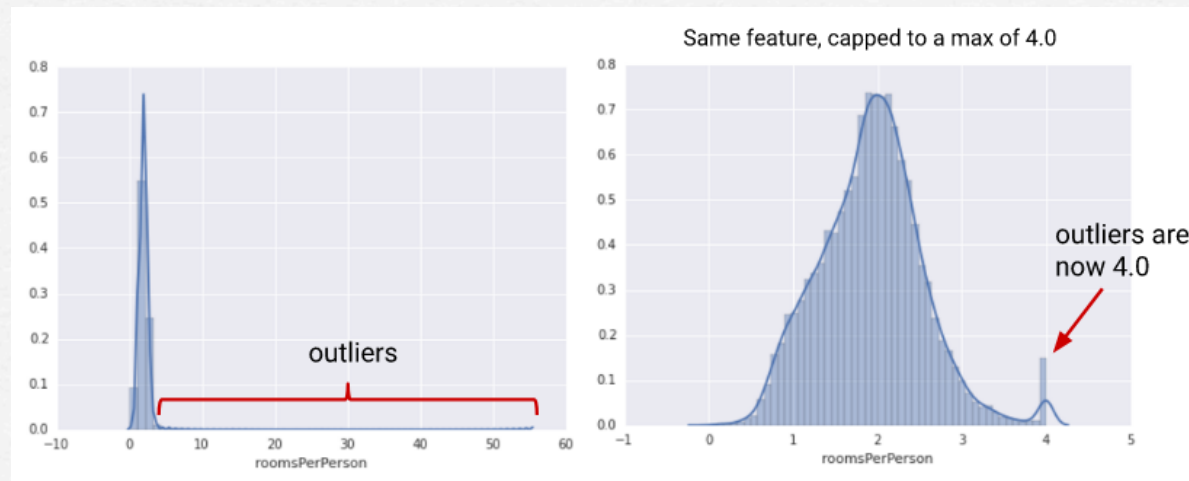
Rescaled into [-1,1] range

But, we should only use the training dataset to compute the minimum and maximum values.



# Data normalization (2)

MinMax scaler faces a problem when the data contains outliers:



We have to first filter-out outliers or if there are not too many outliers use a z-normalization:

where  $\mu$  is the mean and  $\sigma$  the st-dev

$$x' = \frac{(x - \mu)}{\sigma}$$

# Weight initialization

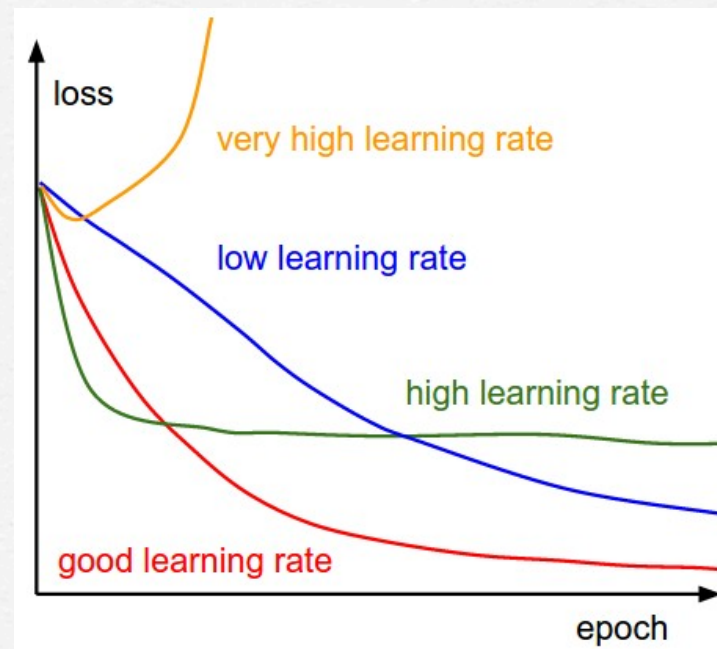
- ❑ **zero initialization**: this places us somewhere in the loss error function we want to minimize. If every time we start in the same place, we may always face the same difficulties to get to the minimum. Moreover, if all weights are equal, all neurons in the first layer will compute the same thing... in conclusion it is a bad idea.
- ❑ **small random numbers**: it has been proposed that weights should be initialized to small random values scaled by a function of the number of inputs ( $n$ ), as follows:

$$w = \text{np.random.randn}(n) * \sqrt{2.0/n}$$



# Effect of learning rate

With low learning rates (blue line) the improvements look linear. With high learning rates they will start to look more exponential. Higher learning rates will decay the loss faster, but they get stuck at worse values of loss (green line).





# Hyper-parameter tuning

- **Rule of thumb**: try several log-spaced values (0.1, 0.01, 0.001) and narrow the grid search to the region where you obtain the lowest validation error.
- For some parameters use the \*3 strategy: e.g., number of hidden neurons: 1, 3, ~10, 30 ...
- Scikit-learn implements **GridSearchCV** and **RandomizedSearchCV**
- Keras implements **KerasTuner** for hyperparameter tuning
- **AutoML**: the process of automating the development of a ML solution

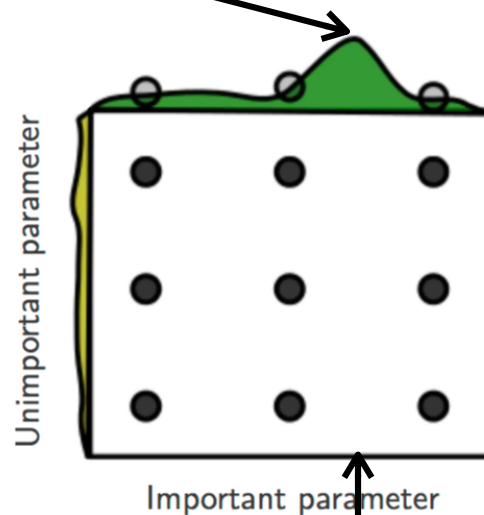
# Benefits of random search

BERGSTRA AND BENGIO

Random Search for Hyper-Parameter Optimization

Best performance

Grid Layout



best hyperparameter value

Bergstra, James, and Yoshua Bengio. "Random search for hyper-parameter optimization."  
Journal of machine learning research 13.2 (2012).

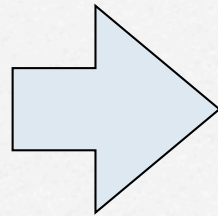
APE 2026



# Model selection



hyper-parameters

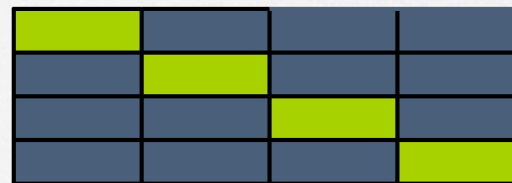
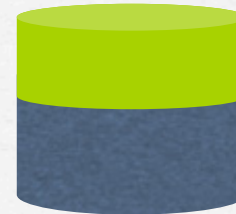


for each config :  
repeat\_several\_times\*  
perform cross-validation

performance

set of model "configs"

- number of layers
- neurons per layer
- learning rate
- momentum term
- number of epochs
- etc...



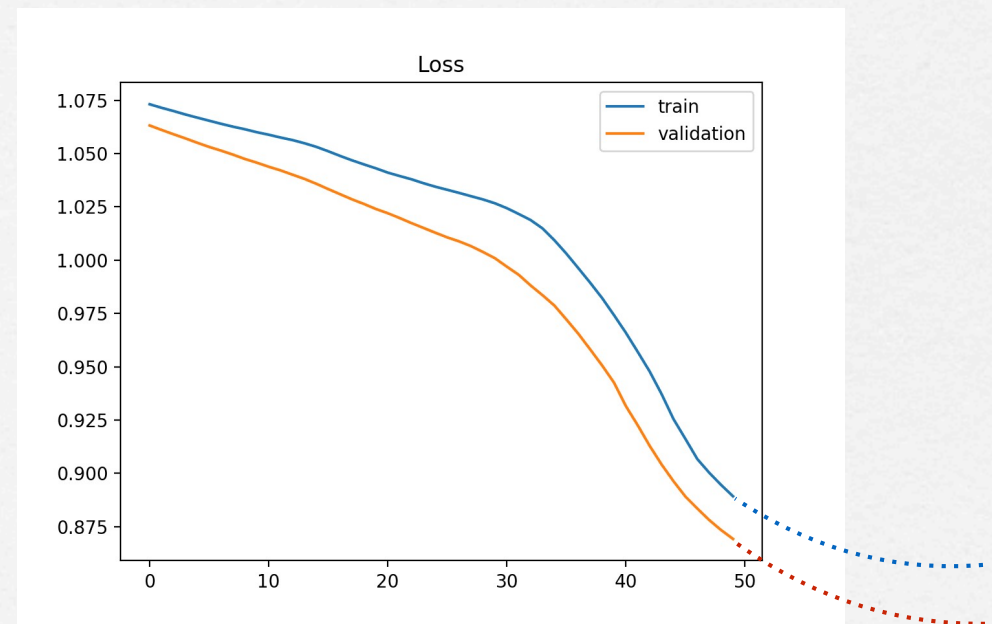
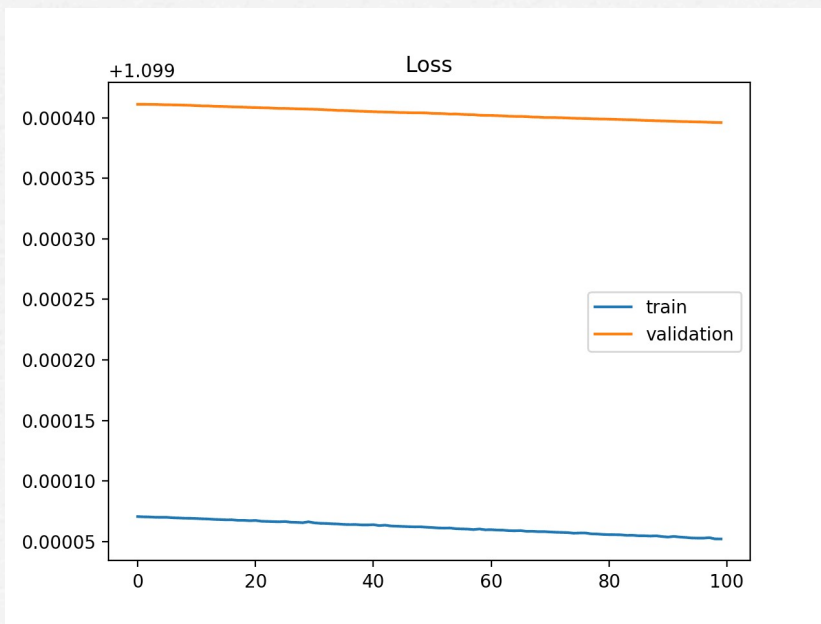
e.g., 4-fold cross-validation



HE<sup>vd</sup>IG\* we may need to try the response of the model for different weight initializations

# Monitoring ANN training (1)

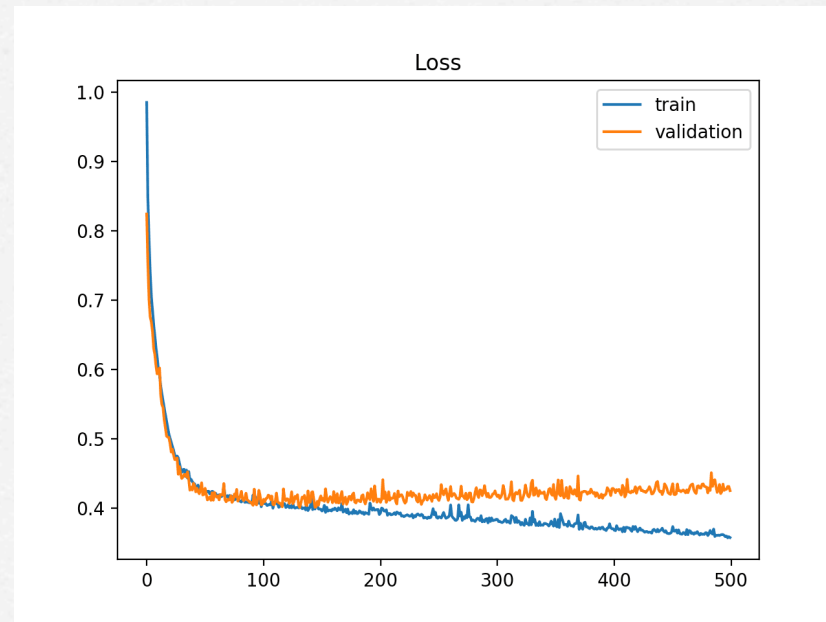
underfitting, probably because the model is too simple (left) or because there is a need for more training iterations (right).





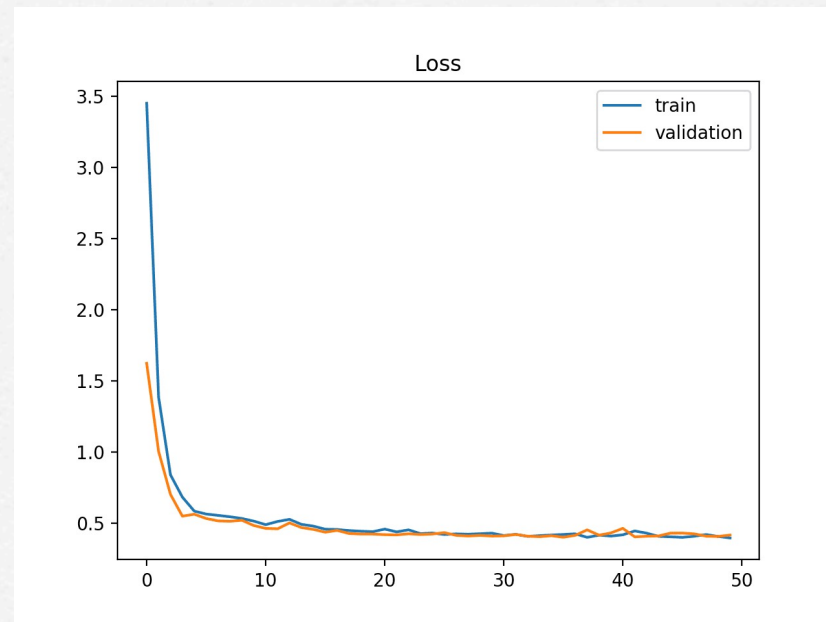
# Monitoring ANN training (2)

loss vs epochs plot: overfitting, probably because the model is too complex; there is an increasing gap between the evolution of the loss evaluated on the training dataset and the loss evaluated on the validation dataset



# Monitoring ANN training (3)

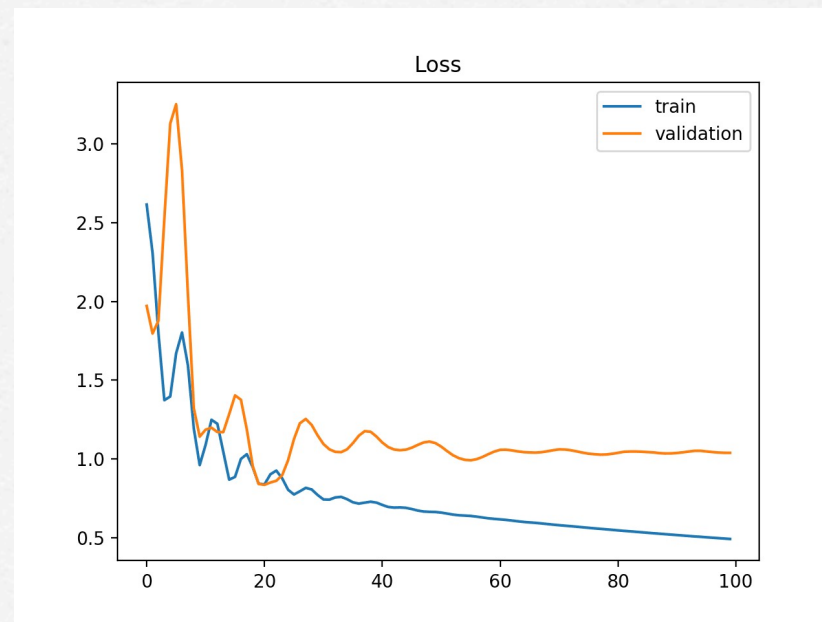
good fit; the loss function reaches a point of stability and there is a small gap between the training and the validation errors





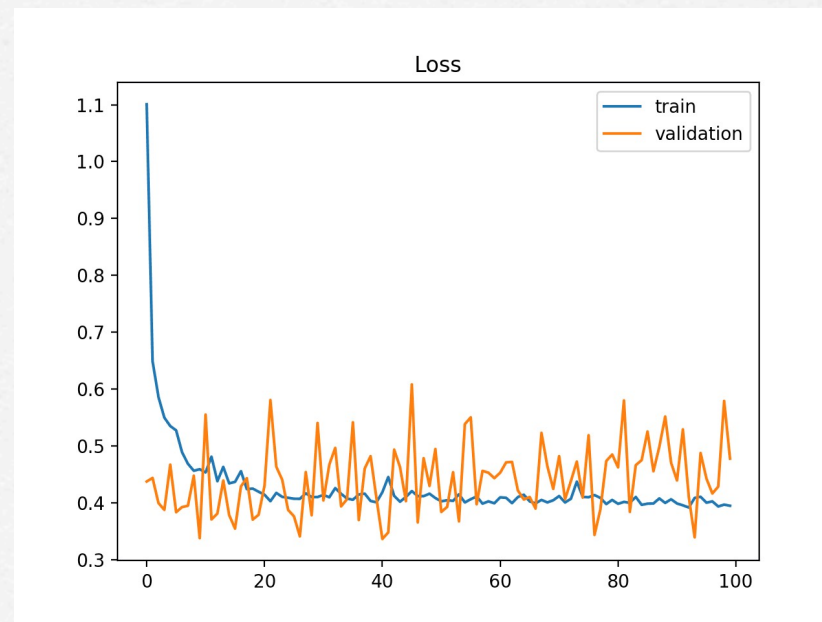
# Monitoring ANN training (4)

Unrepresentative training dataset: it does not provide sufficient information to learn the problem (e.g., too few examples or training data exhibits low diversity)



# Monitoring ANN training (5)

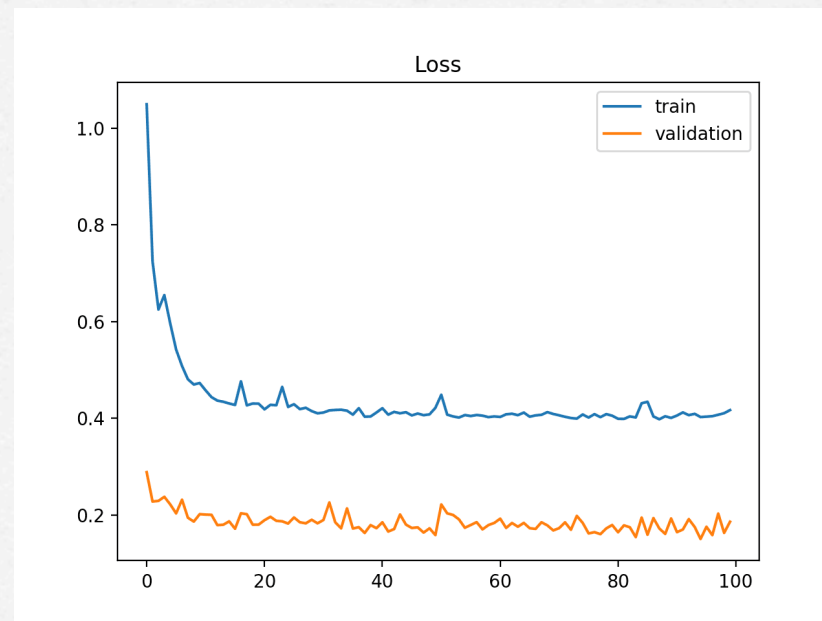
Unrepresentative validation dataset. Here a randomly initialized model (epoch=0) performs similarly (on the validation dataset) to models that have been trained, even after 100 epochs.





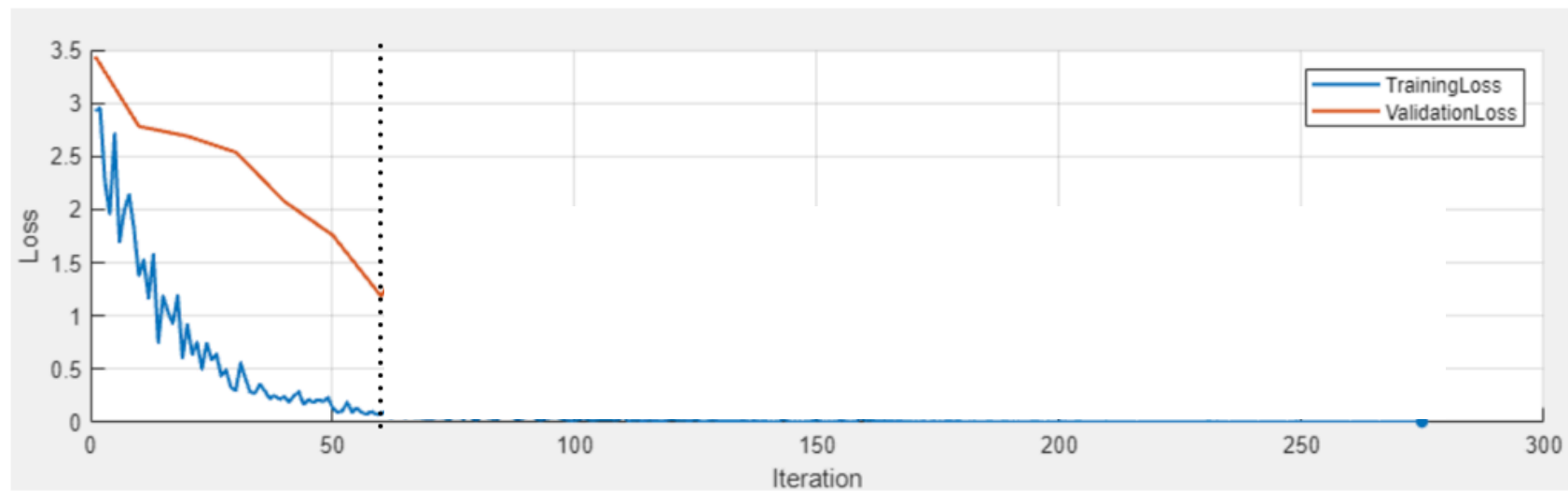
# Monitoring ANN training (6)

Unrepresentative validation dataset. Here, surprisingly the performance of the model on the validation dataset is better than on the training one. This means the validation dataset is too easy compared to the learning problem represented by the training dataset.



# Case study (1)

La courbe ci-dessus illustre l'évolution de la fonction de perte (loss) au cours des 60 premiers epochs lors de l'entraînement d'un réseau de neurones sur un problème de classification, pour les jeux de données d'entraînement et de validation.

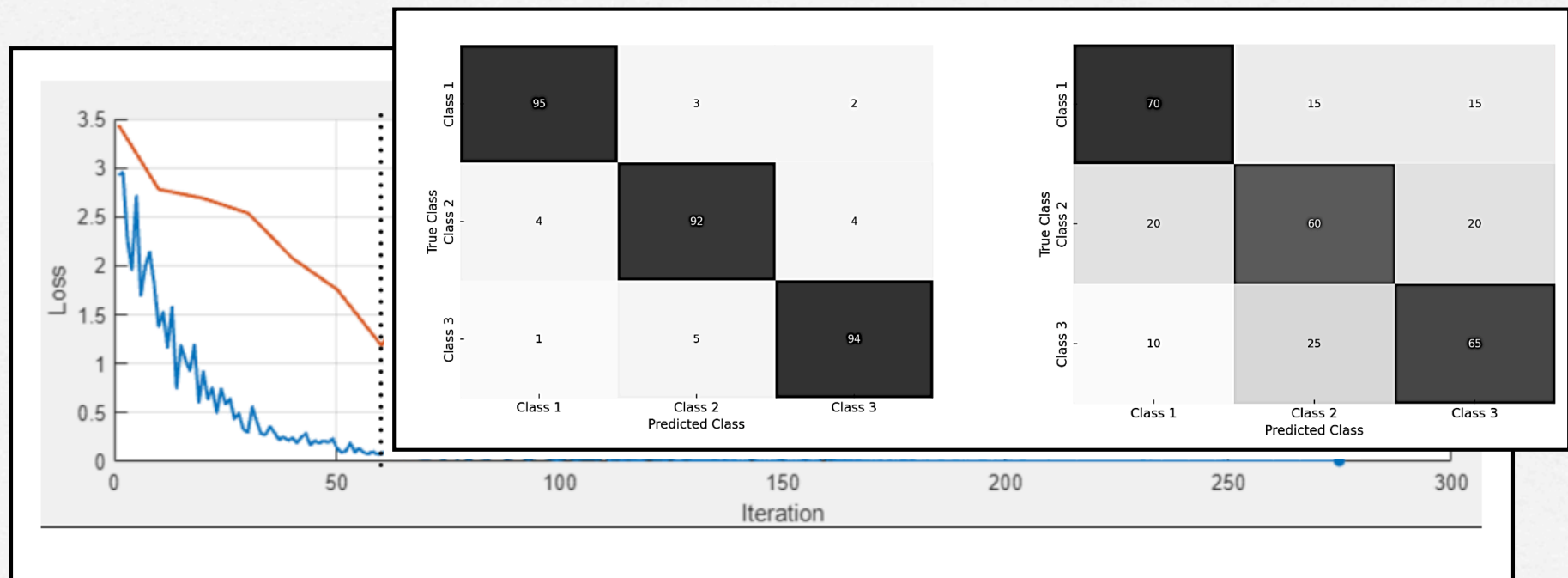




- L'erreur (loss) calculé sur les données d'entraînement est proche de zéro, tandis que l'erreur calculé sur les données de validation est plus grand. On pourrait imaginer qu'il y a un problème d'overfitting et modifier l'architecture du réseau de neurones (p.ex., diminuer le nombre de neurones dans la couche cachée), mais la dynamique d'évolution de la courbe d'erreur de validation montre une possible diminution par la suite (après plus d'epochs).

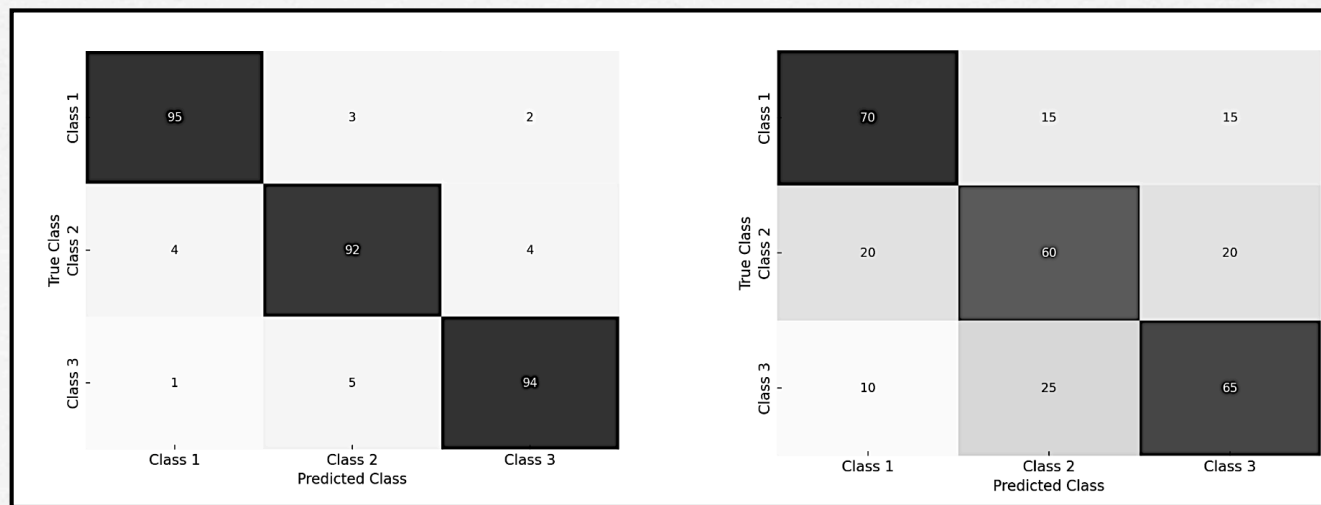
## Case study (2)

La matrice de confusion à gauche montre la performance du modèle après environs 60 itérations (epochs) sur les données d'entraînement. La matrice de confusion à droite illustre la performance de ce modèle lorsqu'il est évalué au même moment avec des données de validation.





- A l'aide des matrices de confusion on confirme la différence de comportement du réseau de neurones (après 60 epochs d'entraînement) face aux données d'entraînement et de validation: il ne généralise pas très bien, car il présente une certaine confusion entre les classes.



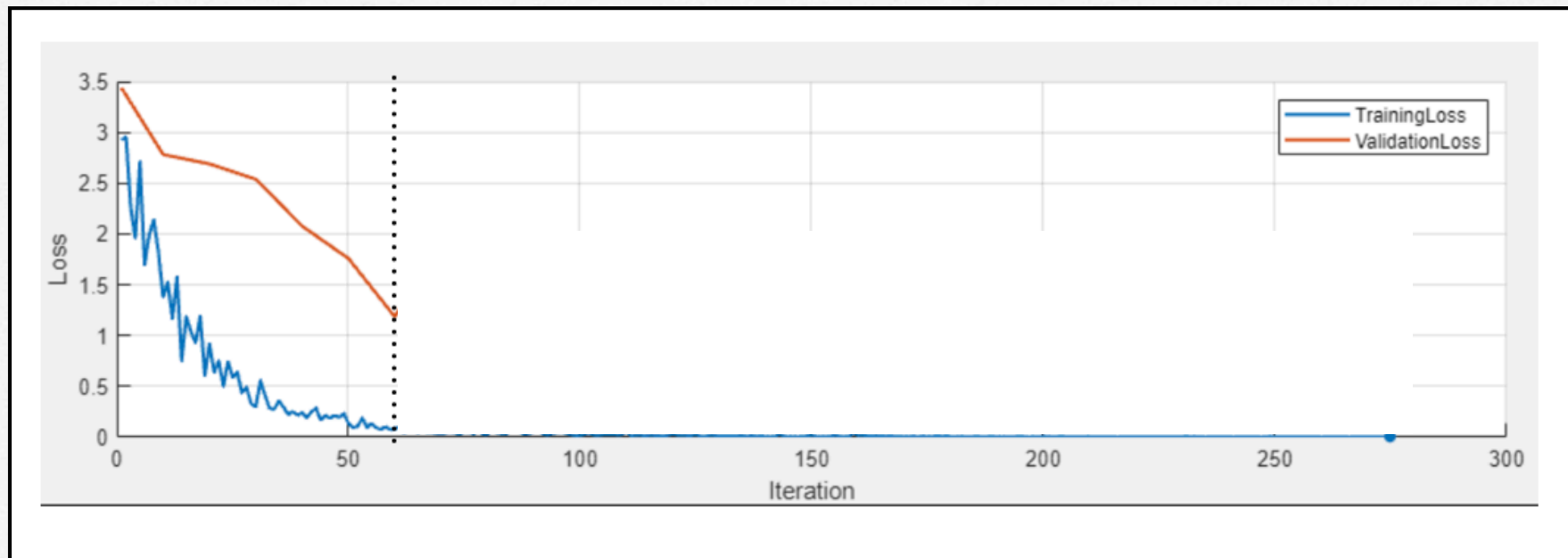
training

validation

# Case study (3)

Quoi faire ?

Supposons qu'on décide de continuer l'entraînement sur plus d'epochs sans changer les hyper-paramètres...





- L'entraînement du réseau de neurones avec les hyperparamètres de base n'a pas donné une bonne suite après les premiers 60 epochs: l'erreur de validation n'a plus diminué comme on l'avait imaginé.
- Deux hypothèses peuvent expliquer ce comportement :
  - (1) l'ensemble de validation n'est pas représentatif, p.ex., il contient peut-être des exemples plus difficiles à classifier ou absents de l'ensemble d'entraînement ; ou
  - (2) le réseau de neurones n'est pas suffisamment complexe. Toutefois, comme le modèle parvient à bien traiter les données d'entraînement, l'hypothèse la plus plausible est la première.